Assignment 3: Build a Security Triage Agent 100 Points Possible

14/10/2026

Add comment

23/09/2026 to 14/10/2026

Details

Objective

Design and build an agentic system that assesses a specific threat or weakness in an infrastructure and assigns an evidence-supported High, Medium, or Low risk rating. Explain the affected or potentially affected assets, rating rationale, uncertainty, and recommended action. The project is graded on the completeness of the design, the working implementation, and the testing.

A weakness may be a software vulnerability, misconfiguration, excessive permission, or exposed credential. Distinguish confirmed findings from questions that require additional evidence. Work as a group and submit one project identifying all members and their contributions.

Select a case and environment

Choose one threat report, vendor advisory, CVE/NVD record, KEV entry, or CTI input. Identify the risk question, cite the source, and save the input used.

Choose the supplied **Windows enterprise environment (https://canvas.duke.edu/courses/83493/files/5480967?wrap=1)** , create your own realistic environment, or use an authorized environment from a group member's current company. You may also adapt the supplied scenario. Provide enough detail about assets, network connections, security controls, and business context to assess applicability and impact. Identify simulated details, assumptions, unknowns, and any changes you make. For a real company, provide a sanitized profile using information you are authorized to share.

DISCLAIMER: T***he supplied Windows Enterprise Environment is an AI-generated fictional teaching environment***; its details may contain simplifications or errors. Validate relevant technical assumptions during your assessment.

Step 1: Design

Using the lessons learned from the in-class design activity, design the architecture to fit this use case. Include a workflow diagram and the following responses, with enough detail for another group member to implement the design:

- Define the risk question, input scope, target environment, and required rating output.
- Create a rating policy for assigning High, Medium, or Low risk using applicability and impact.
- Identify model, code, tool, and human steps; show evidence flow, decision points, and analyst review.
- Explain each tool's purpose, inputs, outputs, source attribution, and role in the risk decision. The examples on the next page are optional; use relevant tools from the list or create your own.
- Create and justify a rating policy based on the applicability of the threat to your environment and the impact it has on your network.
- Show what happens when a lookup fails or a decisive fact is unknown, including when the workflow stops.
- Define a simple acceptance test stating how you will verify that the complete workflow produces a supported rating or an explicit insufficient-evidence status when necessary.

Use the simplest architecture that meets the goal. Document meaningful changes between the design and implementation.

Example tools:

These are examples only feel free to use your own

Example tool	Useful call and role in the risk decision
Tavily web search (https://docs.tavily.com/documentation/api-reference/endpoint/search)	search(query, allowed_domains, max_results) → advisories or service pages. Cite the underlying source; verify date and relevance.
Recorded Future (https://docs.recordedfuture.com/reference/get-started)	enrich(entity_type, value) → threat activity, risk context, references, and timestamps. Explain the effect on the rating. The API key will be supplied separately.
RAG search	search_evidence(query, collection, top_k) → report or environment passages with source IDs and locators.
MITRE ATT&CK (https://attack.mitre.org/resources/attack-data-and-tools/) or **ATLAS (https://atlas.mitre.org/)**	lookup_behavior(query_or_id) → technique definitions and references. Map behavior to access paths and consequences. Use ATLAS for AI threats; mappings carry no risk score.
CVE (https://www.cve.org/About/Overview), **NVD (https://nvd.nist.gov/developers/vulnerabilities)**, and **CVSS (https://www.first.org/cvss/v4.0/user-guide)**	lookup_vulnerability(cve_id) → description, affected versions, references, and available CVSS score/vector/version. CVE identifies a weakness; CVSS describes technical severity.
CISA KEV (https://www.cisa.gov/known-exploited-vulnerabilities-catalog)	lookup_kev(cve_id) → known exploitation and remediation context. Absence from KEV does not establish safety.

Step 2: Implement and test

Implement your design from Step 1. The model must make at least one tool selection or query-construction decision. Your code must execute the request, return the result to the model, and use it in the assessment. Keep source access, tools, model settings, orchestration, rating logic, and validation separately identifiable.

For LLM calls use **Ollama Cloud (https://docs.ollama.com/cloud)**, the **Duke AI Gateway (https://oit.duke.edu/service/ai-gateway/)**, local Ollama, or another accessible model service. Any agent framework is acceptable, including **LangGraph (https://info.langchain.com/langsmith/for-langgraph?utm_campaign=evergreen_langsmith_branded_cv&utm_campaign_id=23553472535&utm_ad_group_id=200117088360&2273731673919&hsa_kw=langgraph&hsa_mt=e&hsa_net=adwords&hsa_ver=3&gad_source=1&gad_campaignid=23553PkieuGn2vfBlsQ3ovuOMPaXeNrN&gclid=Cj0KCQjwzsjVBhC3ARIsALnMv4mHTZjwakDcKa58H_C0O8Ht1vADuBhlh5y-Xczhq_Qp9ar1xCTTgEsaApFDEALw_wcB)**, **CrewAI (https://crewai.com/)**, **Pi (https://pi.dev/)**, or your own implementation.

1. Save a structured assessment

Any clear format (e.g. JSON) containing these fields is acceptable:

- Asset and threat: the asset/service ID and the threat or weakness being assessed.
- Decision: High, Medium, or Low when supported; otherwise an explicit insufficient-evidence or review status.
- Rationale: applicability, impact, other factors used, and the policy rule or criteria applied.
- Evidence and uncertainty: source references/locators, dates, facts, assumptions, unknowns, confidence, and whether evidence is live, cached, or simulated.
- Action: recommended next step and evidence that would change or resolve the decision.

2. Demonstrate a complete run

Demonstrate one complete run. Preserve inputs, configuration, tool results, and output. Use live lookups when available and label fallbacks. Verify that:

- Intake loads the selected threat or weakness and the correct environment snapshot.
- The model requests an allowed tool with valid arguments; code executes it and returns attributed evidence to the model.
- Environment lookup and at least one relevant external evidence lookup execute and contribute to the risk assessment.
- The agent compares threat prerequisites with environment facts, preserves unknowns, applies the group's policy, and produces a supported rating or explicit insufficient-evidence status with a recommended action.
- A group member verifies the assessment and its cited evidence.

3. Evaluate the outputs

- Compare the output risk rating to your groups consensus. Does the output rating match the policy Record expected versus actual results for these three small policy checks. Set expected outcomes from your group's documented policy before running the checks:
- Test one controlled failure, such as an unavailable API or invalid tool request. Verify appropriate handling. Record expected versus actual results and limitations.
- (Optional) Tracing is optional for Assignment 2. Formal evaluations and systematic iteration will be in future work. Save your Assignment 2 inputs, environment snapshot, configuration, code version, and outputs as a baseline.

Step 3 Submit

Submit one GitHub repository link accessible to the instructor or one ZIP file through Canvas. Identify all members, their contributions. Include:

- README with setup, dependency versions, configuration without secrets, and instructions to run the complete workflow.
- Completed or revised in-class architecture activity, workflow diagram, tool contracts/examples, and the group's justified and implemented applicability-and-impact rating policy.
- Selected case and source references, the supplied or group-created synthetic environment or an authorized sanitized company profile, and permitted supporting data.
- Runnable code, final assessment, complete-run demonstration checklist, expected versus actual results for the three policy checks and one controlled failure, remaining limitations, the saved Assignment 2 baseline, and disclosure of AI assistance.

If your group does not present in class, submit a video demo/walkthrough as a file or instructor-accessible link. Show the design, complete workflow, tool calls, rating rationale, evidence, and limitations.

Rubric

Criterion	Points	Full credit evidence
Design completeness	35	Completed in-class architecture activity and diagram define the risk question, environment, inputs, a justified policy using applicability and impact, tool contracts, evidence flow, uncertainty, failure paths, human review, and success tests with enough detail to implement. Source choices are justified by the risk decision.
Implementation	45	Working agent executes model-driven tool use, queries environment and relevant external evidence, and implements the group's applicability-and-impact policy to produce a supported rating or documented insufficient-evidence status. The assessment cites evidence, distinguishes severity from local risk, preserves unknowns, and recommends action. GitHub repository or ZIP is reproducible and identifies member contributions.
Testing	20	Complete workflow demonstrated in class or by video; expected versus actual results for applicability-change, impact-change, and missing/conflicting-evidence policy checks plus one controlled failure check; evidence that outputs and citations were reviewed; documented limitations and saved baseline for Assignment 4.
Total	100	

(https://canvas.duke.edu/courses/83493/modules/items/1005348)

(https://canvas.duke.edu/courses/83493/modules